AOSP 进阶丛书 · 第十八册 · ANDROID 13 (API 33)

CTS 认证实战指南

CTS 结构（200-300 万断言）、Tradefed 调度引擎、VTS HAL 验证、厂商适配实战
——Google Play 预装的唯一通行证。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：兼容性的唯一权威 2
第一章 CTS 的结构与覆盖面 2
第二章 Tradefed：测试调度引擎 2
第三章 VTS：HAL 的验证 3
第四章 开发者视角：怎么适配 3
第五章 设计决策五问 4
附录 A 实战命令 5
附录 B 快问快答（10 条） 5
附录 C 自检题 5
第六章 CTS 模块解剖：从 Android.bp 到 CtsOsTestCases.apk 6
第七章 运行链：从 cts-tradefed 命令到 instrumentation 10
第八章 计划体系与过滤：plan、include 链与 known failures 15
第九章 Host 侧测试、结果体系与 Retry 19
教科书习题集（分级） 24
术语表 26
排障手册：CTS 认证问题定位 28
CTS 核心速查表 30

本册速览

CTS 结构：cts/tests/tests/ 下数千测试用例
Tradefed：测试调度框架（device selection/test execution/result aggregation）
VTS：HAL 验证（书14 的 AIDL 接口逐个过）
测试流程：下载 CTS→跑→提交 Google 审→认证
设计动机：为什么 CTS 是兼容性的唯一权威

前言：兼容性的唯一权威

CTS（Compatibility Test Suite）是 Google 认证一台 Android 设备的唯一标准——**通过 CTS = 拿到 GMS（Google Play）预装权**。本册讲 CTS 的结构（测什么）、Tradefed 怎么跑、VTS（HAL 验证）与 CTS 的关系、以及开发者视角的"怎么适配"。

源码基线不变。主角：`cts/`（测试集）、`cts/tools/tradefed/`（调度框架）。

第一章 CTS 的结构与覆盖面

> **设计洞察 · 学习目标**
> 1. 能说出 cts/tests/tests/ 的目录组织规律与单个用例的构成（Test Class+AndroidManifest）；2. 能解释 CTS 断言的本质是"framework 行为符合 SDK 文档"；3. 能估算 CTS 的断言规模与单机运行时长。

text cts/tests/tests/（本树实际布局）

```
├── app/（Activity/Service/Notification…数千用例）
├── app.usage/ · accounts/ · animation/ · database/ · location/…
├── net/ · os/ · permission/ · security/ ├── telephony/ · view/ · widget/
└── … 按模块分组
每个用例 = junit Test Class + AndroidManifest（声明 targetSdk）
// 讲解注释：CTS 的核心断言是"framework API 的行为符合 SDK 文档"——
// 任何 deviation 都是 fail。
```

1.1 一个测试用例的样子

java cts/tests/tests/app/src/android/app/cts（节选+讲解注释）

```
// 典型 CTS 用例的结构（以 ActivityTest 为例）
@SmallTest
public class ActivityTest extends ActivityInstrumentationTestCase2<MyActivity> {
public void testActivityLifecycle() {
// 讲解注释①：启动 Activity→断言 onCreate→onStart→onResume 顺序
// 讲解注释②：断言基于 android.test 框架（junit3 风格为主）
// 讲解注释③：通过 = 行为与 SDK 文档一致
}
}
// 讲解注释④：一个 CTS 包约 200-300 万个断言，
// 跑完一台设备 8-72 小时（取决于模块）。
```

第二章 Tradefed：测试调度引擎

设计洞察 · 学习目标

1. 能画出 TestInvocation→设备选择→执行→结果聚合的四段流水线；2. 能说出 Tradefed 独立发行（cts.zip 内预构建）的原因与形态；3. 能使用 run cts 系列命令完成全量/单模块/单用例三种粒度的执行。

text cts/tools/tradefed/host/src/…（结构+讲解注释）

```
TestInvocation（入口：解析命令→选设备→跑→聚合结果）
├── DeviceMonitor（设备状态：USB/温度/存储）
├── TestRunner（执行器：逐用例跑→捕获失败→截图/日志）
├── ResultReporter（报告：XML/HTML 每用例结果）
└── RetryFilter（重试：已知 flaky 的自动重跑）
// 讲解注释：Tradefed 是独立可执行 jar（cts-tradefed 命令）——
// 不依赖 AOSP 构建，预构建发行（cts.zip 内）。
```

2.1 实战流程

bash

```
# 下载 CTS（Google 官方发布，非源码编译）
# 解压→运行：
cts-tradefed run cts --plan CTS -s <serial> # 跑完整 CTS
cts-tradefed run cts -m CtsAppTest -s <serial> # 只跑 app 模块
# 结果：testResult.xml + screenshots/ + logcat/ → 提交 Google 审
```

第三章 VTS：HAL 的验证

设计洞察 · 学习目标

1. 能说出 CTS/VTS/GTS 三者在被测契约层与发行方上的分工；2. 能解释为什么书14 的 AIDL HAL 必须过 VTS；3. 能把本册与书7 的面试题互相对应。

VTS（Vendor Test Suite）验证 HAL 接口实现正确性——**书14 写的 HAL 必须过 VTS**：

text VTS 与 CTS 的分工

```
CTS：framework API 行为（App 视角）
VTS：HAL 接口（书14 AIDL HAL 的每个方法调用是否按规范返回）
└── 本树 system/vts/ + test/vts/
GTS：Google 专有（GMS 验证——闭源发布）
```

第四章 开发者视角：怎么适配

设计洞察 · 学习目标

1. 能按"改了什么"快速判断要过哪个套件（CTS/VTS/GTS）；2. 能列出四类高频 fail 场景与对应根因；3. 能走通"失败→定位→修复或 waiver"的完整处置路径。

场景	要过什么	常见 fail 原因
厂商 HAL	VTS	返回值不符合 .aidl 定义/超时
定制 framework	CTS	行为偏离 SDK 文档
改了 SELinux 策略	CTS+VTS	avc denied 导致服务起不来
GMS 预装	CTS+GTS	所有通过后才拿 Play Store 权

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释CTS 认证的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

> **设计洞察 · 面试视角："CTS 过不了怎么办"**
> 定位：testResult.xml 找 fail 用例→看截图/日志→分析是 framework 偏离还是测试环境 flaky→修代码或申请 waiver（Google 审批豁免）。

第五章 设计决策五问

> **设计洞察 · 学习目标**
> 1. 能复述五个"为什么"并用自己的话给出理由链；2. 能把每条设计决策与具体源码机制（shard、RetryFilter、分套件发行）对应起来；3. 能在面试中用 30 秒讲清任何一条。

- **为什么 CTS 是唯一权威而非文档？** 文档可被误读；代码断言=机器可验证。CTS 通过=所有 App 在你设备上行为正确（Play Store 的信任前提）。
- **为什么 Tradefed 独立于 AOSP 构建？** 发行节奏不同（CTS 每月新版与 Android 版本绑定）；测试框架闭源演进。
- **为什么 VTS 与 CTS 分开？** 覆盖层不同（HAL vs framework）；厂商定制主要影响 HAL——分开发行更精准。
- **为什么 CTS 那么久？** 200-300 万断言+逐用例等待 UI 渲染+网络超时容忍——可在多台设备分片并行（shard）。
- **为什么 flaky 用例需要 RetryFilter？** UI 测试受设备状态影响（温度/存储/网络波动）——已知 flaky 重跑一次降低误报。

附录 A 实战命令

目标	命令	耗时
完整 CTS	cts-tradefed run cts --plan CTS	8-72h
单模块	cts-tradefed run cts -m CtsAppTest	分钟级
单测试	cts-tradefed run cts -t android.app.cts.ActivityTest#testLifecycle	秒级
重跑失败	cts-tradefed run cts --retry <session_id>	智能
VTS	vts-tradefed run vts -m VtsHalPowerV1_0Target	分钟级
结果	查看 testResult.xml / results/<session>/	—

附录 B 快问快答（10 条）

- CTS 全称？——Compatibility Test Suite。
- CTS 通过意味着什么？——可以拿 GMS 预装权（Play Store）。
- CTS 大约多少断言？——200-300 万。
- Tradefed 是什么？——测试调度框架（选设备→跑→聚合）。
- VTS 测什么？——HAL 接口（CTS 测 framework API）。
- GTS 是什么？——Google 专有（GMS 验证）。
- CTS 跑多久？——8-72 小时（可分片并行）。
- fail 了怎么办？——分析 testResult.xml→修代码或申请 waiver。
- 谁跑 CTS？——设备厂商（在 Google 实验室或自建环境）。
- App 开发者需要过 CTS 吗？——不需要（CTS 测 framework 行为）。

附录 C 自检题

- CTS/VTS/GTS 三者的覆盖面与发行方。
- Tradefed 的执行流水线四段。
- 厂商"改了 HAL 忘了过 VTS"的后果链。

第六章 CTS 模块解剖：从 Android.bp 到 CtsOsTestCases.apk

设计洞察 · 学习目标

1. 能读懂一个 CTS 模块的 Android.bp 构建定义并说出每个关键字段的作用；2. 能解读模块的 AndroidTest.xml 配置（preparer/test/module_controller 三段结构）；3. 能说出设备侧用例的注解（@AppModeFull 等）如何影响模块在计划中的执行方式；4. 能用 TEST_MAPPING 把模块接入提交前验证。

本节在第一章"测什么"与第二章"怎么调度"之间补上"一个模块从何而来"。前两章把 CTS 当作整体，本章拆开一个真实的模块：`cts/tests/tests/os/`，构建产物叫 **CtsOsTestCases.apk**。它是第六到第九章反复使用的解剖标本——后续运行链（第七章）、计划过滤（第八章）、结果与 Retry（第九章）都以它为例。

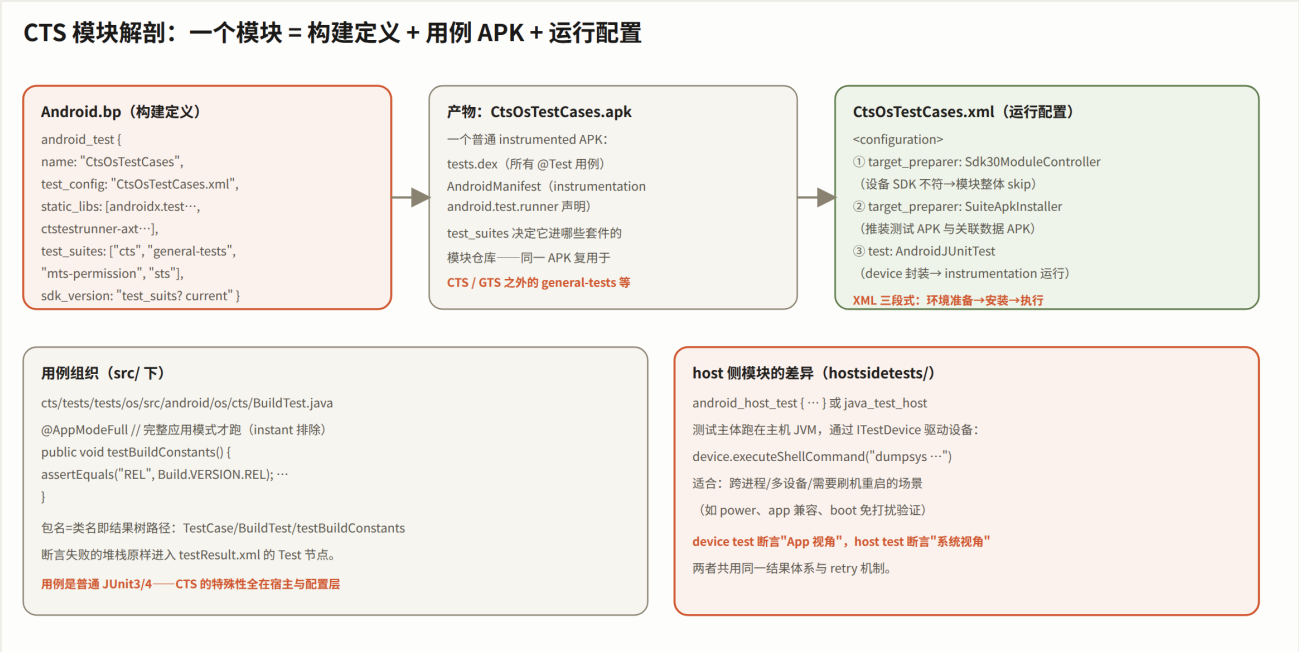


图 1 模块解剖三部曲：Android.bp 构建定义 → CtsOsTestCases.apk 产物 → AndroidTest.xml 运行配置三段式

6.1 模块在源码树中的位置与构建定义

链路位置：Soong 构建系统读取 `cts/tests/tests/os/Android.bp` → 产出 `CtsOsTestCases.apk`（设备侧）+ `CtsOsTestCases.xml`（模块配置，随 APK 一起进入 `android-cts/testcases/` 目录）。第七章会看到 Tradefed 正是从这个目录发现模块的。

java cts/tests/tests/os/Android.bp:19-83（节选+讲解注释）

```
android_test {
name: "CtsOsTestCases", // 讲解注释①：android_test 模板 = 编译成设备上安装的测试 APK
defaults: [
"cts_defaults", // 讲解注释②：CTS 全家共享的默认警告/SDK 设置
```

```
"mts-target-sdk-version-current", // 讲解注释③：同一份代码还挂进 MTS（Mainline
套件，第三章提到过分套件思想）
],
compile_multilib: "both", // 讲解注释④：32/64 位各装一份跑一遍——对应第八章的 multi_abi 参数化
static_libs: [
"androidx.test.core",
"androidx.test.ext.junit",
"androidx.test.rules",
"androidx.test.uiautomator",
"compatibility-device-util-axt", // 讲解注释⑤：CTS 设备侧公共工具库（cts/common/device-side
编译而来）
"ctstestrunner-axt", // 讲解注释⑥：CTS 定制 runner 支持库，配合 androidx 的 AndroidJUnit
Runner
"testng",
"truth-prebuilt",
"guava",
"junit",
"Harrier", // 讲解注释⑦：bedstead 多用户测试框架，配置文件里按注解排除它
…
],
jni_libs: [ // 讲解注释⑧：os 模块含 JNI 用例（内存/进程用例要碰底层）
"libcts_jni",
"libctsos_jni",
…
],
srcs: [
"src/**/*.java",
"src/**/*.kt", // 讲解注释⑨：Java 与 Kotlin 混编——Android 13 的 CTS 已大量 Kotli
n 化
"src/android/os/cts/IParcelFileDescriptorPeer.aidl", // 讲解注释⑩：Binder
用例需要跨进程服务，AIDL 一起编进 AP
K
…
],
// Set its own test config to prevent sharing with cts-platform-version-check
test_config: "CtsOsTestCases.xml", // 讲解注释⑪：显式指定模块配置，防止同名模块互相"抢"默认配置
// Tag this module as a cts test artifact
test_suites: [ // 讲解注释⑫：test_suites 决定这个模块被哪些套件收录
"cts", // cts → 出现在 android-cts/testcases/
"general-tests", // general-tests → Atest/CI 通用跑
"mts-permission", // mts-permission → 只在 Mainline 模块验证时跑
"sts", // sts → 安全补丁套件（与第八章 cts-exclude 的 AsbSecurityTest
排除呼应）
],
sdk_version: "test_current", // 讲解注释⑬：test_current 才能看见 @TestApi——CTS
测的就是系统暴露面
min_sdk_version: "29",
data: [ // 讲解注释⑭：辅助 APK（AutoRevoke 三个受测 App），由配置里的 FilePusher
推到设备
":CtsAutoRevokeQApp",
…
],
per_testcase_directory: true, // 讲解注释⑮：每个用例独立的临时目录，避免用例间文件残留互相污染
}
```

```
// 讲解注释⑯：一个 android_test = APK + 配置 + 归属套件三件事，这是所有 300+ 个 CTS
设备侧模块的共同模板。
```

关键点：test_suites 是"归属声明"——它把构建产物登记进对应套件的发布清单；test_config 是"行为声明"——告诉 Tradefed 怎么跑。两者分离正是第一章"按模块分组"的实现机制。

6.2 模块配置 AndroidTest.xml：preparer → test → 收尾

链路位置：第七章的调度引擎（ITestSuite）加载每个模块时，先解析这份 XML——它是 Tradefed 配置（configuration）对象在磁盘上的形态。注意它和第八章的 plan XML 是两种粒度：模块配置描述"单模块怎么跑"，计划配置描述"哪些模块跑"。

xml cts/tests/tests/os/CtsOsTestCases.xml:16-56（全文+讲解注释）

```
<configuration description="Configuration for OS Tests">
<option name="test-suite-tag" value="cts" /> // 讲解注释①：归属 cts
套件——套件级排除规则靠它生效
<option name="config-descriptor:metadata" key="component" value="framework" /> //
讲解注释②：统计口径——失
败按 framework 组件归类
<option name="config-descriptor:metadata" key="parameter" value="instant_app" /> //
讲解注释③：声明可参
数化——
<option name="config-descriptor:metadata" key="parameter" value="multi_abi" /> //
讲解注释③：这四种家
族的模块会被展开成
<option name="config-descriptor:metadata" key="parameter" value="secondary_user" />//
讲解注释③：CtsO
sTestCases[instant] 等多个变体（第八章）
<option name="config-descriptor:metadata" key="parameter" value="multiuser" />
<object type="module_controller"
class="com.android.tradefed.testtype.suite.module.Sdk30ModuleCo
ntroller" /> // 讲解注释④：模块控制器——只在 API≥30 的设备上跑本模块，否则整体 skip
<target_preparer class="com.android.tradefed.targetprep.suite.SuiteApkInstaller"> //
讲解注释⑤：阶段一
target_preparer——把 APK 安装进设备
<option name="cleanup-apks" value="true" />
<option name="test-file-name" value="CtsOsTestCases.apk" />
</target_preparer>
<test class="com.android.tradefed.testtype.AndroidJUnitTest" > // 讲解注释⑥：阶段二
test——真正的执行器（IRemo
teTest，第七章主角）
<option name="package" value="android.os.cts" /> // 讲解注释⑦：instrumentation 只跑 and
roid.os.cts 包
<option name="runtime-hint" value="3m15s" /> //
讲解注释⑧：预估耗时——分片（shard）调度用它均衡负
载
<option name="runner" value="androidx.test.runner.AndroidJUnitRunner" /> // 讲解注释⑨：设备侧
runne
r 类名
<option name="exclude-annotation" value="com.android.bedstead.harrier.annotations.RequireRun
OnWorkProfile" /> // 讲解注释⑩：过滤注解——带这些注解的用例在本模块不跑
<option name="exclude-annotation" value="com.android.bedstead.harrier.annotations.RequireRun
OnSecondaryUser" />
</test>
```

```
<!-- Create Place to store apks -->
<target_preparer class="com.android.tradefed.targetprep.RunCommandTargetPreparer"> //
讲解注释⑪：阶段三
——直接执行 adb 命令做环境准备
<option name="run-command" value="mkdir -p /data/local/tmp/cts/os" />
<option name="run-command" value="am wait-for-broadcast-idle" /> //
讲解注释⑫：等广播队列空闲——减少环境噪声导致
的 flaky
<option name="teardown-command" value="rm -rf /data/local/tmp/cts/os" /> //
讲解注释⑬：teardown
收尾，可逆性是 preparer 的纪律
</target_preparer>
<!-- Load additional APKs onto device -->
<target_preparer class="com.android.compatibility.common.tradefed.targetprep.FilePusher"> //
讲解
注释⑭：把 Android.bp 的 data 字段产物推到设备
<option name="push" value="CtsAutoRevokeSApp.apk->/data/local/tmp/cts/os/CtsAutoRevokeSApp.a
pk" />
…
</target_preparer>
</configuration>
// 讲解注释⑮：记住三段结构：module_controller（跑不跑）→ target_preparer（环境）→
test（执行）——排障手册的 NotExecuted 场景就出在第一、二段。
```

6.3 设备侧用例长什么样：BuildTest 走读

链路位置：AndroidJUnitTest 通过 instrumentation 拉起 APK 后，androidx.test.runner.AndroidJUnitRunner 反射执行这些 junit 类。看一个最小的真实用例（比第一章的骨架更完整）。

java cts/tests/tests/os/src/android/os/cts/BuildTest.java:40-81（节选+讲解注释）

```
public class BuildTest extends TestCase { // 讲解注释①：junit3 风格 TestCase——CTS
大量存量用例的风格基线
private static final String RO_PRODUCT_CPU_ABILIST = "ro.product.cpu.abilist";
private static final String RO_PRODUCT_CPU_ABILIST32 = "ro.product.cpu.abilist32";
private static final String RO_PRODUCT_CPU_ABILIST64 = "ro.product.cpu.abilist64";
/**
* Verify that the values of the various CPU ABI fields are consistent.
*/
@AppModeFull(reason = "Instant apps cannot access APIs") // 讲解注释②：声明本用例不能在 instant
app 参数变体下跑——
public void testCpuAbi() throws Exception { // 讲解注释②：第八章的参数化展开会尊重这个注解
runTestCpuAbiCommon();
if (android.os.Process.is64Bit()) { // 讲解注释③：运行时探测 ABI 位宽，分支断言
runTestCpuAbi64();
} else {
runTestCpuAbi32();
}
}
/**
* Verify that the CPU ABI fields on device match the permitted ABIs defined by CDD.
*/
public void testCpuAbi_valuesMatchPermitted() throws Exception { //
讲解注释④：CDD（兼容性定义文档）的条目直接代码化
```

```
// The permitted ABIs are listed in https://developer.android.com/ndk/guides/abis.
Set<String> just32 = new HashSet<>(Arrays.asList("armeabi", "armeabi-v7a", "x86"));
Set<String> just64 = new HashSet<>(Arrays.asList("x86_64", "arm64-v8a"));
Set<String> all = new HashSet<>();
all.addAll(just32);
all.addAll(just64);
Set<String> allAndEmpty = new HashSet<>(all);
allAndEmpty.add("");
// The cpu abi fields on the device must match the permitted values.
assertValueIsAllowed(all, Build.CPU_ABI); // 讲解注释⑤：断言 =
"SDK/文档承诺的取值集合"——第一章说的
// CPU_ABI2 will be empty when the device does not support a secondary CPU architecture.
assertValueIsAllowed(allAndEmpty, Build.CPU_ABI2); // 讲解注释⑤：'任何 deviation 都是 fail'
在这里落地
// The supported abi fields on the device must match the permitted values.
assertValuesAreAllowed(all, Build.SUPPORTED_ABIS);
assertValuesAreAllowed(just32, Build.SUPPORTED_32_BIT_ABIS);
assertValuesAreAllowed(just64, Build.SUPPORTED_64_BIT_ABIS);
}
// 讲解注释⑥：testCpuAbi_common 段还会用 getprop 读系统属性并与 Build.* 交叉验证——
// "属性世界"与"API 世界"必须一致，这正是第四章'行为偏离 SDK 文档'的一类具体形态。
```

6.4 TEST_MAPPING：同一个模块在提交流水线中的用法

模块不只服务认证，也服务开发者的日常。`cts/tests/tests/os/TEST_MAPPING`（JSON）声明：presubmit 阶段跑 `CtsOsTestCases`，但**排除** `LargeTest` 注解用例与两个已知慢/环境敏感用例（`BuildTest#testSdkInt`、`StrictModeTest#testNonSdkApiUsage`）。对照第八章你会认出：这同样是 include/exclude 过滤，只是宿主从 cts-tradefed 换成了 Atest 与 Gerrit 流水线——**同一份模块资产，多套过滤视角复用**，这是 CTS 工程学的核心经济学。

> **设计洞察 · 设计洞察：为什么模块=APK+XML 双产物**
>
> Google 没有把"怎么跑"写进构建系统，而是让 Soong 只负责"是什么"（APK），Tradefed 配置负责"怎么跑"。这样测试框架可以独立演进（第五章"Tradefed 独立于 AOSP 构建"的落地形态），模块作者也能在不重编 APK 的情况下调整 preparer 与过滤。代价是 XML 与代码可能漂移——所以第七章会看到有一个 presubmit 单测专职校验全量配置可加载。

第七章 运行链：从 cts-tradefed 命令到 instrumentation

> **设计洞察 · 学习目标**
>
> 1. 能从启动脚本逐行说出 cts-tradefed 的运行环境准备与入口主类；2. 能画出 CompatibilityConsole→ITestSuite→模块加载→IRemoteTest→instrumentation 的五级执行链并标注每级职责；3. 能解释 shard 分片与 token 机制如何把 8-72 小时压缩到多机并行；4. 能说出 presubmit 单测如何守护全量模块配置的可加载性。

第二章给了 Tradefed 的鸟瞰图，本章把"选设备→跑→聚合"展开为一条可以在源码里逐站验证的执行链。先声明一个重要事实：**本树中 Tradefed 核心以预编译 jar 发行**（`tools/tradefederation/prebuilts/filegroups/` 下的 `tradefed.jar` 与 `compatibility-tradefed.jar`），Java 源码不在树内——但所有类名、jar 分层、配置解析行为都可以在本树内交叉验证（启动脚本、Android.bp、配置 XML、以及 CTS 自己的单测）。以下每一站都给出本树内的锚点。



图 2 运行链六段：控制台→计划叠加→模块切分→设备执行→结果汇聚，下接 retry 闭环与失败形态分类

7.1 第一站：启动脚本做了什么

链路位置：你在终端敲下 `cts-tradefed`，执行的就是 `cts/tools/cts-tradefed/etc/cts-tradefed` 这个 shell 脚本。它是 java 进程的"产科医生"。

sh cts/tools/cts-tradefed/etc/cts-tradefed:39-119（节选+讲解注释）

```
checkPath aapt // 讲解注释①：入口三件套依赖检查——缺任何一个直接退出而非半路诡异失败
checkPath adb
checkPath java
# check java version
JAVA_VERSION=$(java -version 2>&1 | grep -m 1 'version [ "]\(1\.8\|9\|11\).*[ "]') //
讲解注释②：锁定 JDK
8/9/11——
if [ "${JAVA_VERSION}" == "" ]; then // 讲解注释②：Tradefed 的反射与序列化代码对新 JDK
敏感，宁可挡在门外
echo "Wrong java version. 1.8, 9, or 11 is required."
exit
fi
# check if in Android build env
if [ ! -z "${ANDROID_BUILD_TOP}" ]; then // 讲解注释③：两种运行形态之一——源码构建环境
if [ ! -z "${ANDROID_HOST_OUT}" ]; then
CTS_ROOT=${ANDROID_HOST_OUT}/cts // 讲解注释④：make cts 之后的产物根
else
```

```
CTS_ROOT=${ANDROID_BUILD_TOP}/${OUT_DIR:-out}/host/${OS}/cts
fi
if [ ! -d ${CTS_ROOT} ]; then
echo "Could not find $CTS_ROOT in Android build environment. Try 'make cts'"
exit
fi;
fi;
if [ -z ${CTS_ROOT} ]; then
# assume we're in an extracted cts install
CTS_ROOT="$(dirname $(realpath $0))/../.." // 讲解注释⑤：形态之二——Google 发布的 android-cts 解压包（厂商认证的主形态）
fi;
if [ -z ${JAVA_BINARY} ]; then
JAVA_BINARY=${CTS_ROOT}/android-cts/jdk/bin/java // 讲解注释⑥：优先用 CTS 自带 JDK——认证环境拒绝"碰运气"
fi
JAR_DIR=${CTS_ROOT}/android-cts/tools
for JAR in ${JAR_DIR}/*.jar; do // 讲解注释⑦：tools/ 下所有 jar 全部进 classpath——包括 tradefed.jar 与
JAR_PATH=${JAR_PATH}:${JAR} // 讲解注释⑦：compatibility-tradefed.jar（分层见 7.2）
done
# include any host-side test jars
for j in $(find ${CTS_ROOT}/android-cts/testcases -name '*.jar'); do // 讲解注释⑧：host 侧模块的 jar（第九章的 Ja
rHostTest）也在此进入 classpath
JAR_PATH=${JAR_PATH}:$j
done
${JAVA_BINARY} $RDBG_FLAG -Xmx6g -XX:+HeapDumpOnOutOfMemoryError -cp ${JAR_PATH}
-DCTS_ROOT=${CTS_RO
OT} com.android.compatibility.common.tradefed.command.CompatibilityConsole "$@"
// 讲解注释⑨：真正的主类 CompatibilityConsole；-Xmx6g + OOM 时自动 heap dump——排障手册的 host 侧 OOM 场景证据链从这里开始
```

类名验证：`CompatibilityConsole` 类确实存在于 `compatibility-tradefed.jar` 中（`unzip -l` 可列出 `com/android/compatibility/common/tradefed/command/CompatibilityConsole.class`）。第六章 `cts-tradefed/Android.bp` 里 `tradefed_binary_host { name: "cts-tradefed", wrapper: "etc/cts-tradefed", version: "13_r1" }` 把脚本与本 jar 绑定成最终发行物。

7.2 第二站：jar 分层与五级执行链

`cts-tradefed/Android.bp` 声明了三层依赖，对应三次"泛化→特化"：

层	构建名	职责	代表类
Tradefed 核心	tradefed（预编译 jar）	通用调度：配置/设备/shard/结果	ITestSuite、AndroidJUnitTest、IRemoteTest
套件通用层	compatibility-tradefed	一切兼容性套件（CTS/VTS/MTS）共有的语义	CompatibilityTestSuite、CompatibilityConsole、RetryFilterHelper
CTS 特化层	cts-tradefed-harness	CTS 专属资源与计划 XML	res/config 下的全部计划配置

五级执行链如下（每级都给出本树可验证的锚点）：

1. **Console 级**：`CompatibilityConsole` 解析 `run cts --plan CTS` 命令，找到第八章讲到的计划配置。

2. **Suite 级**：计划配置最终实例化为一个 `ITestSuite` 实现——CTS 里是 `com.android.compatibility.common.tradefed.testtype.suite.CompatibilityTestSuite`（jar 内已验证存在）。它枚举 `android-cts/testcases/` 下的全部模块配置，借助 `CompatibilitySuiteModuleLoader` 把每个 XML 变成可执行模块。

3. **Shard 级**：`ITestSuite` 把模块清单交给 `com.android.tradefed.invoker.shard.ShardHelper`/`StrictShardHelper` 切分，产出 N 个子 invocation 由 `TestsPoolPoller` 在多台设备上认领；`runtime-hint`（第六章⑧）就是这里的均衡依据；`LastShardDetector` 负责收尾聚合。token 维度（`shard/token/TokenProperty`、`TelephonyTokenProvider`）让"需要 SIM 卡的模块"只落到插了 SIM 的分片上。

4. **Test 级**：每个模块配置里的 `<test>` 对象实现 `IRemoteTest` 接口（`com.android.tradefed.testtype.IRemoteTest`）。设备侧模块几乎都用 `AndroidJUnitTest`，它实现 `IShardableTest` 与 `ITestFilterReceiver`——后者正是第八章 exclude-filter 能注入的契约接口。

5. **Instrumentation 级**：`AndroidJUnitTest` 组装 `com.android.ddmlib.testrunner.RemoteAndroidTestRunner`，最终执行形如 `am instrument -w -e package android.os.cts androidx.test.runner.AndroidJUnitRunner` 的设备命令，经 ddmlib 逐用例回传状态。

> **设计洞察 · 设计洞察：为什么链的尽头是 IRemoteTest 这个小接口**
> IRemoteTest 只要求一个 run(TestInformation, ITestInvocationListener) 方法。定义了最小契约，GTest、JarHostTest、MoblyBinaryHostTest（7.3 的 runner 白名单里全部在列）都能塞进同一条流水线，shard 与结果聚合对"测试是什么"完全无感。这回答了第二章"调度引擎"为何能同时调度 CTS/VTS——接口下沉到了极限。

7.3 第三站：谁在守护这条链——CTS 的配置宪法

CTS 用一个 host 侧单测把"所有模块配置必须可加载、必须用受支持的 runner"变成 presubmit 硬约束。这份文件同时是 7.2 各类名真实性的最佳证人。

java cts/tools/cts-tradefed/tests/src/com/android/compatibility/common/tradefed/presubmit/CtsConfigLoadingTest.java:137-163,195-243（骨架+讲解注释）

```
private static final Set<String> SUPPORTED_CTS_TEST_TYPE = new HashSet<>(Arrays.asList( //
讲解注释
①：runner 白名单——宪法正文
// Cts runners
"com.android.compatibility.common.tradefed.testtype.JarHostTest", // 讲解注释②：host 侧 runner（第九章）
"com.android.compatibility.testtype.DalvikTest",
"com.android.compatibility.testtype.LibcoreTest",
```

```
"com.drawelements.deqp.runner.DeqpTestRunner", // 讲解注释③：连 GPU 一致性测试家族的 runner
也名列其中
// Tradefed runners
"com.android.tradefed.testtype.AndroidJUnitTest", // 讲解注释④：第六章解剖的 CtsOsTestCases
用的就是它
"com.android.tradefed.testtype.ArtRunTest",
"com.android.tradefed.testtype.HostTest",
"com.android.tradefed.testtype.GTest",
"com.android.tradefed.testtype.mobly.MoblyBinaryHostTest"
));
/**
* In Most cases we impose the usage of the AndroidJUnitRunner because it supports all the
* features required (filtering, sharding, etc.). We do not typically expect people to need a
* different runner.
*/
private static final Set<String> ALLOWED_INSTRUMENTATION_RUNNER_NAME = new HashSet<>(); //
讲解注释
⑤：设备侧 runner 名单同样受限——javadoc 明说理由：过滤与 sharding 特性
static {
ALLOWED_INSTRUMENTATION_RUNNER_NAME.add("android.support.test.runner.AndroidJUnitRunner");
ALLOWED_INSTRUMENTATION_RUNNER_NAME.add("androidx.test.runner.AndroidJUnitRunner");
}
/**
* Test that configuration shipped in Tradefed can be parsed.
* -> Exclude deprecated ApkInstaller.
* -> Check if host-side tests are non empty.
*/
@Test
public void testConfigurationLoad() throws Exception { // 讲解注释⑥：全量扫描
android-cts/testcases/ 下每
个 .config
String ctsRoot = System.getProperty("CTS_ROOT");
File testcases = new File(ctsRoot, "/android-cts/testcases/");
…
// We expect to be able to load every single config in testcases/
for (File config : listConfigs) {
IConfiguration c = ConfigurationFactory.getInstance()
.createConfigurationFromArgs(new String[] {config.getAbsolutePath()}); // 讲解注释⑦
：与真实运行同一套解析器
if (c.getDeviceConfig().size() > 2) {
throw new ConfigurationException(String.format("%s declares more than 2 devices.", c
onfig));
}
int deviceCount = 0;
for (IDeviceConfiguration dConfig : c.getDeviceConfig()) {
// Ensure the deprecated ApkInstaller is not used anymore.
for (ITargetPreparer prep : dConfig.getTargetPreparers()) {
if (prep.getClass().isAssignableFrom(ApkInstaller.class)) {
throw new ConfigurationException( // 讲解注释⑧：废弃 API 禁用——SuiteApkInstall
er 强制替代
String.format("%s: Use com.android.tradefed.targetprep.suite."
+ "SuiteApkInstaller instead of com.android.compatibility."
+ "common.tradefed.targetprep.ApkInstaller, options will be
"
+ "the same.", config));
```

```
}
…
}
deviceCount++;
}
// 讲解注释⑦：后续还会校验 IRemoteTest 类名在白名单、host 侧测试非空，以及
MANDATORY_PARAMETERS_FAMILY——
// instant/多 abi/多用户三大家族必须在模块配置里显式声明（呼应第八章 8.4）——一份 447
行的"配置宪法"。
```

设计洞察 · 排障视角：读懂这条链的最小价值

设备上报"模块 NotExecuted"时，沿链定位：Console 能起（脚本报错看 7.1）→ Suite 加载失败（看 7.3 解析异常）→ preparer 失败（看第六章 6.2 阶段一/三）→ test 报设备失联（看排障手册场景一）。四站对应四种完全不同的修复路径——链路思维比背结论重要。

第八章 计划体系与过滤：plan、include 链与 known failures

设计洞察 · 学习目标

1. 能说出"计划=可叠加的配置片段"这一核心抽象并追出 cts 计划的完整 include 链；2. 能区分 exclude-filter 与 test-arg 两种过滤的作用层次；3. 能解释 known failures 文件的格式、粒度与它在 waiver 流程中的位置；4. 能描述参数化模块（instant/多 abi/多用户）如何从一条 metadata 声明展开成多个执行变体。

第二章说 `cts-tradefed run cts --plan CTS`，本章回答：**"CTS 计划"到底是什么**。答案藏在 `cts/tools/cts-tradefed/res/config/`——目录里 40 多个 XML，每个都是一块可复用的配置，计划就是把它们 include 成一棵树。

8.1 计划即配置叠加：cts 计划的完整 include 链

链路位置：`CompatibilityConsole`（第七章第一站）解析 `--plan cts` 时，按名字在 jar 资源里找到 `cts.xml`，递归展开 include，最终合成一份运行时配置交给 `CompatibilityTestSuite`。先看计划本体——它小得惊人：

xml cts/tools/cts-tradefed/res/config/cts.xml:16-24（全文+讲解注释）

```
<configuration description="Runs CTS from a pre-existing CTS installation">
<include name="cts-common" /> // 讲解注释①：公共底座——前置检查、报告器、套件标签全在这层
<include name="cts-exclude" /> // 讲解注释②：排除层——不属于主 CTS 的用例（见 8.2）
<include name="cts-exclude-instant" /> // 讲解注释③：instant 变体专用排除层
<option name="plan" value="cts" /> // 讲解注释④：给自己贴 plan 标签——结果记录与 --plan 匹配用
</configuration>
// 讲解注释⑤：整个计划只有 9 行。复杂度全部下沉到被 include 的片段——计划是"组合"，不是"堆料"。
```

继续展开 `cts-common.xml`，这里定义了一次 CTS 运行的"物理环境"：

xml cts/tools/cts-tradefed/res/config/cts-common.xml:16-65（骨架+讲解注释）

```
<configuration description="Common configuration for cts and cts-reference-aosp">
<include name="everything" /> // 讲解注释①：Tradefed 全局默认（设备发现、日志、重试策略的兜底值）
<option name="compatibility:run-suite-tag" value="cts" /> //
讲解注释②：套件标签——只有模块配置声明同一标签（第六章①）才会被收
录
<!-- Enable module parameterization to run instant_app modules in main CTS -->
<option name="compatibility:enable-parameterized-modules" value="true" /> //
讲解注释③：参数化总开关（8.4）
<!-- Main CTS executes both single and multi-devices in the same plan during sharding -->
<option name="multi-devices-modules" value="RUN" /> // 讲解注释④：多设备模块与单设备模块同场分片
…
<include name="cts-preconditions" /> // 讲解注释⑤：运行前体检——user
构建、语言、锁屏、WiFi（8.3）
<include name="cts-system-checkers" /> // 讲解注释⑤：系统状态检查器
<include name="cts-known-failures" /> // 讲解注释⑥：已知失败排除表——与 waiver
流程直接挂钩（8.2）
<option name="test-tag" value="cts" />
<option name="enable-root" value="false" /> // 讲解注释⑦：认证纪律——CTS 不允许 root 设备
<!-- retain 200MB of host log -->
<option name="max-log-size" value="200" /> // 讲解注释⑧：日志上限——host 侧磁盘保护
<!-- retain 200MB of logcat -->
<option name="max-tmp-logcat-file" value="209715200" />
<target_preparer class="com.android.tradefed.targetprep.RunCommandTargetPreparer"> //
讲解注释⑨：计划级
也能挂 preparer——临时关掉包校验器
<option name="run-command" value="settings put global package_verifier_enable 0" />
<option name="teardown-command" value="settings put global package_verifier_enable 1"/>
</target_preparer>
<target_preparer class="com.android.compatibility.common.tradefed.targetprep.PropertyCheck">
//
讲解注释⑩：三项体检：user 构建/美式英语/非测试笼模式
<option name="property-name" value="ro.build.type" />
<option name="expected-value" value="user"/> <!-- Device should have user build -->
<option name="throw-error" value="false"/> <!-- Only print warning if not user build -->
</target_preparer>
<target_preparer class="com.android.compatibility.common.tradefed.targetprep.PropertyCheck">
//
讲解注释⑪：语言体检——CTS 对 locale 有强假设
<option name="property-name" value="ro.product.locale" />
<option name="expected-value" value="en-US"/> <!-- Device locale should be US English -->
<option name="throw-error" value="false"/> <!-- Only print warning if not en-US -->
</target_preparer>
<target_preparer class="com.android.compatibility.common.tradefed.targetprep.PropertyCheck">
//
讲解注释⑫：测试笼模式体检——这项直接 throw-error
<option name="property-name" value="persist.sys.test_harness" />
<option name="expected-value" value="false"/> <!-- Device shouldn't be in test harness mode
-->
<option name="throw-error" value="true"/>
</target_preparer>
<template-include name="reporters" default="basic-reporters" /> //
讲解注释⑬：模板含——报告器可在命令行整体替换，不必改
XML
</configuration>
```

```
// 讲解注释⑭：注意 include 的顺序语义：后 include 的片段可以覆盖先前的
option——叠加不是拼接，是"逐层赋值"。
```

叠加关系总结成一张表：

片段	承担的职责	被谁引用
cts.xml	计划入口，三段 include	命令行 --plan cts
cts-common.xml	套件标签/参数化开关/体检/日志上限/报告器	cts.xml
cts-exclude.xml、cts-exclude-instant.xml	主计划排除规则	cts.xml
cts-preconditions.xml、cts-system-checkers.xml、cts-known-failures.xml	前置条件与已知失败	cts-common.xml
cts-dev.xml	再 include cts，跳过体检与参数化，仅主 ABI	开发自测

`cts-dev.xml` 的关键三行印证了叠加语义：`<option name="skip-preconditions" value="true" />`、`<option name="compatibility:primary-abi-only" value="true" />`、`<option name="compatibility:enable-parameterized-modules" value="false" />`——**同一个模块集合，因为 include 了一个不同片段，运行体积缩小一个数量级**。这就是第五章"为什么 CTS 那么久"的工程答案：全量慢，但组合可快。

8.2 过滤的三个层次：exclude-filter、test-arg 与 known failures

Tradefed 的过滤注入依赖 `ITestFilterReceiver` 契约（第七章 7.2 第五级），CTS 套件层把它升级成三种写法：

xml cts/tools/cts-tradefed/res/config/cts-exclude.xml:16-40（全文+讲解注释）

```
<configuration description="Excluded tests from main CTS runs">
<include name="cts-developer-exclude" />
<!-- b/64127136 -->
<option name="compatibility:exclude-filter" value="CtsSecurityHostTestCases
android.security.cts
.SELinuxHostTest#testNoExemptionsForBinderInVendorBan" /> //
讲解注释①：粒度四级：模块/包/类/方法——这里是"模块+类#方法"
<option name="compatibility:exclude-filter" value="CtsSecurityHostTestCases
android.security.cts
.SELinuxHostTest#testNoExemptionsForSocketsBetweenCoreAndVendorBan" />
<option name="compatibility:exclude-filter" value="CtsSecurityHostTestCases
android.security.cts
.SELinuxHostTest#testNoExemptionsForVendorExecutingCore" />
<!-- Test Harness Mode tests are not a part of CTS. They are a part
of their own testing plan, as they reset the device during the
test. It's possible and ideal in the future to incorporate the
tests into CTS, but until then, they should be excluded. -->
<option name="compatibility:exclude-filter" value="CtsTestHarnessModeTestCases" /> //
讲解注释②：整模块
排除——注释说明理由：它会重启设备
```

```
<!-- Exclude downstreaming tests from CTS, i.e. tests added after the
first major release for this API level (They are pulled into GTS
instead). -->
<option name="compatibility:test-arg"
value="com.android.tradefed.testtype.AndroidJUnitTest:excl
ude-annotation:com.android.compatibility.common.util.CtsDownstreamingTest" /> //
讲解注释③：test-arg
语义：对指定 runner 注入选项——按注解排除而非按名字排除
<option name="compatibility:test-arg"
value="com.android.compatibility.common.tradefed.testtype.
JarHostTest:exclude-annotation:com.android.compatibility.common.util.CtsDownstreamingTest" />
/
/ 讲解注释④：host 侧 runner 也要注入一遍——两条 runner 走两条注入线
<!-- Exclude @SecurityTest tests from CTS. These are tested in STS. b/180417031-->
<option name="compatibility:test-arg"
value="com.android.tradefed.testtype.AndroidJUnitTest:excl
ude-annotation:android.platform.test.annotations.AsbSecurityTest" /> //
讲解注释⑤：安全用例划给 STS 套件——与第
六章 test_suites 里的 sts 闭环
</configuration>
// 讲解注释⑥：exclude-filter 是"名单制"（点名到方法），test-arg
是"标注制"（按注解批量）——名单精确但会腐烂，标注可维护但需要用例自律。
```

而 known failures 是排除名单的"官方登记簿"：

xml cts/tools/cts-tradefed/res/config/cts-known-failures.xml:17-33（节选+讲解注释）

```
<configuration description="Configuration with CTS known failures" >
<!-- <option name="compatibility:exclude-filter" value="MODULE_NAME" /> Excludes whole module
--
>
<!-- <option name="compatibility:exclude-filter" value="MODULE_NAME PACKAGE_NAME" /> Excludes
wh
ole package -->
<!-- <option name="compatibility:exclude-filter" value="MODULE_NAME PACKAGE_NAME.CLASS_NAME"
/>
Excludes whole class -->
<!-- <option name="compatibility:exclude-filter" value="MODULE_NAME
PACKAGE_NAME.CLASS_NAME#TEST
_NAME" /> Excludes individual test --> // 讲解注释①：文件头自文档化——四种粒度一目了然
<!-- b/38182235 -->
<option name="compatibility:exclude-filter" value="CtsLocationTestCases
android.location.cts.Gns
sTtffTests#testTtffWithNetwork" /> // 讲解注释②：每条排除必须有 bug 号——可追溯是纪律
<option name="compatibility:exclude-filter" value="CtsLocationTestCases[instant]
android.locatio
n.cts.GnssTtffTests#testTtffWithNetwork" /> // 讲解注释③：[instant]
后缀——参数化变体要单独登记（8.4）
<!-- b/23776893 -->
<option name="compatibility:exclude-filter" value="CtsDumpsysHostTestCases
android.dumpsys.cts.D
umpsysHostTest#testBatterystatsOutput" /> // 讲解注释④：host 侧模块同样在此登记（第九章讲 host
侧）
…
</configuration>
// 讲解注释⑤：本树该文件 288 行、约 151 条过滤——每条 = 一个已确认的环境型失败。它就是第五章
RetryFilter 之外的"白名单豁免"。
```

known failures 与 waiver 的关系（呼应第四章）：厂商在认证时遇到 Google 已确认的 flaky/环境失败，无需修设备——这些用例已在计划加载时被排除，结果 XML 里不会出现。反过来，若你的失败**不在** known failures 且稳定复现，那就是框架偏离，要么修代码要么走 waiver 申请——Google 审批通过后才会进入下一版 known failures。排除表因此成为"认证公平性"的账本：每个条目都可追溯到 bug。

8.3 Preconditions：跑之前的体检科

`cts-preconditions.xml` 在计划展开时注册一串 target_preparer（第六章 6.2 认识的阶段一角色）：`StayAwakePreparer` 保持屏幕常亮、`SettingsPreparer` 关掉错误对话框（`hide_error_dialogs=1`）避免 UI 用例被弹窗干扰、`WifiCheck` 确认连接、`ApkPreconditionCheck` 拉起 `CtsPreconditions.apk` 做设备侧深度检查、`DeviceInfoCollector` 采集设备信息写进结果。`cts-common.xml` 的三个 `PropertyCheck`（user 构建、en-US 语言、非 test_harness 模式）也在此层。**体检不过 = 整个计划 NotExecuted**——这是排障手册场景二的根因来源。

8.4 参数化模块：一条声明展开成 N 次执行

第六章 6.2 的 CtsOsTestCases.xml 声明了四种 `parameter` metadata。开启 `enable-parameterized-modules` 后，`ITestSuite` 加载模块时会查这份声明，用 `com.android.tradefed.testtype.suite.params.ModuleParameters` 家族（jar 内已验证 `InstantAppHandler`、`SecondaryUserHandler`、`FoldableHandler` 等）把一个模块克隆成多个变体：instant 变体以 instant app 方式安装运行、multi_abi 变体对 32/64 位各跑一遍、secondary_user 变体切换用户执行。变体的名字带方括号后缀（`CtsOsTestCases[instant]`），这正是 8.2 known failures 里要单独登记 `[instant]` 变体的原因——**排除的匹配单位是变体而非模块**。参数不匹配的用例靠 `@AppModeFull` 之类注解（第六章 6.3 注释②）自我声明退出。`cts-dev` 计划关闭参数化，用牺牲覆盖面换开发迭代速度。

> **设计洞察 · 设计洞察：为什么过滤要做成配置片段而非命令行参数**
> 命令行参数只属于一次运行，配置片段属于套件的生命周期。exclude-filter/test-arg/known-failures 三者全部以 XML 片段形式进入 include 链，意味着每次 Google 发布新版 CTS，这些"认证政策"本身也随套件版本化——厂商复现认证结果时用的是同一份政策快照，而不是各自粘贴的命令行。这是把"公平"做成构建产物。

第九章 Host 侧测试、结果体系与 Retry

> **设计洞察 · 学习目标**
> 1. 能说出 host 侧模块与设备侧模块在构建类型、配置 runner、测试基类上的三点差异；2. 能写出 testResult.xml 的五层结构并用脚本从中提取失败清单；3. 能解释 Retry 命令的"加载旧结果→过滤→重跑"三步与其中跳过成功用例的机制；4. 能判断一次失败该走 Retry、known failures 还是 waiver。

前几章的主角都是设备侧用例。但 CTS 里有一整类测试**根本不在设备上运行**——它们在 host 机器上用 adb 检查设备。本章同时收拢两条线：host 侧测试的形态，以及一切运行最终沉淀的产物——testResult.xml 与 Retry 机制。

9.1 Host 侧模块：java_test_host 与 JarHostTest

链路位置：第六章设备侧模块用 `android_test` 构建 APK；host 侧模块用 `java_test_host` 构建 **host jar**，运行时由套件层注入命令。以 `cts/hostsidetests/dumpsys/`（第一章提过的 dumpsys 验证家族）为例：

java cts/hostsidetests/dumpsys/Android.bp:19-34（全文+讲解注释）

```
java_test_host {
name: "CtsDumpsysHostTestCases", // 讲解注释①：java_test_host = 编译成 host 机器上的
jar，不进设备
defaults: ["cts_defaults"],
srcs: ["src/**/*.java"],
// Must match the package name in CtsTestCaseList.mk
libs: [
"cts-tradefed", // 讲解注释②：依赖套件通用层——第七章 7.2 的三层分层在这里兑现
"tradefed", // 讲解注释③：依赖 Tradefed 核心——host 代码直接 import ITestDevice 等
"truth-prebuilt",
],
// tag this module as a cts test artifact
test_suites: [ // 讲解注释④：归属与设备侧模块完全同一套机制
"cts",
"general-tests",
],
}
// 讲解注释⑤：对比第六章 android_test：没有 sdk_version/静态库打包——它运行在 host JVM，不需要
APK 化。
```

对应模块配置 `cts/hostsidetests/dumpsys/AndroidTest.xml` 只有三个要点：`test-suite-tag=cts`（同第六章）；metadata 声明 `not_instant_app/not_multi_abi`（系统服务 dump 与 instant/多 abi 无关，**负向参数声明**让参数化引擎跳过这些变体——第八章 8.4 的另一面）；`<test class="com.android.compatibility.common.tradefed.testtype.JarHostTest">` 指定 host runner 并用 `<option name="jar" value="CtsDumpsysHostTestCases.jar" />` 告诉它反射扫描哪个 jar——第七章 7.3 runner 白名单的第一个名字就是它。

host 用例本体（抽掉 license 头）：

java cts/hostsidetests/dumpsys/src/android/dumpsys/cts/BaseDumpsysTest.java:19-56（节选+讲解注释）

```
import com.android.compatibility.common.tradefed.build.CompatibilityBuildHelper; //
讲解注释①：host 侧也能拿
到套件文件目录（取辅助 APK）
import com.android.ddmlib.testrunner.RemoteAndroidTestRunner; // 讲解注释②：ddmlib 同样是
host 侧武器——host 用
例可以反过来拉起设备侧 instrumentation
import com.android.tradefed.device.CollectingOutputReceiver;
import com.android.tradefed.device.DeviceNotAvailableException;
```

```
import com.android.tradefed.device.ITestDevice; // 讲解注释③：ITestDevice = "带超时与恢复语义的
adb 门面"，host 用例的
第一入口
import com.android.tradefed.log.LogUtil.CLog;
import com.android.tradefed.result.CollectingTestListener; // 讲解注释④：收集 instrumentation
结果供 host 断言
import com.android.tradefed.testtype.DeviceTestCase; // 讲解注释⑤：注意——host 用例继承的是
DeviceTestCase（junit
3）而非 BaseHostJUnit4Test
import com.android.tradefed.testtype.IBuildReceiver;
public class BaseDumpsysTest extends DeviceTestCase implements IBuildReceiver { //
讲解注释⑥：IBuildRece
iver 注入构建信息（找 jar/产物用）
protected static final String TAG = "DumpsysHostTest";
private static final String TEST_RUNNER = "androidx.test.runner.AndroidJUnitRunner";
/**
* A reference to the device under test.
*/
protected ITestDevice mDevice; // 讲解注释⑦：host 用例持有设备句柄——同一测试可顺序操作多台设备
protected IBuildInfo mCtsBuild;
@Override
protected void setUp() throws Exception {
super.setUp();
mDevice = getDevice(); // 讲解注释⑧：基类统一取设备——子类只写断言逻辑
}
// 讲解注释⑨：与设备侧的本质差异：设备侧断言跑在设备进程；host 侧断言跑在 host JVM，
// 通过 executeShellCommand/instrumentation 操纵设备——可以验证"设备自己看不见的东西"（如 dumpsys
全局输出）。
```

具体到子类 `GfxInfoDumpsysTest#testGfxinfoFramestats`（同目录）：先 `uninstallPackage` 清场→`CompatibilityBuildHelper.getTestFile` 取辅助 APK 并安装→`executeShellCommand("am start -W ...")` 拉起被测应用→`dumpsys gfxinfo ... framestats` 抓帧数据→在 host 上解析 `---PROFILEDATA---` 段逐行校验→finally 再卸载。**"准备设备→抓输出→host 解析"三段式**是所有 dumpsys/procstats 类 host 用例的通用节奏。

> **设计洞察 · 设计洞察：什么时候必须用 host 侧测试**
>
> 设备进程只能看见自己的世界：申请到的权限、自己的 dumpsys 视图。而"系统是否泄漏了别的应用的 Activity 栈""SELinux 拒绝后服务是否仍在""dumpsys 输出格式是否符合 CDD"都必须站在设备之外观察。第三章 VTS 的 HAL 接口测试同理站在设备之外——host 侧能力是分套件（CTS/VTS/GTS）共同的技术底座，差异只在被验证的契约。

9.2 结果体系：testResult.xml 的五层结构

一次运行结束后，`results/<session-id>/` 目录沉淀 `testResult.xml`（部分工具链称 `test_result.xml`）。树里有两个真实消费者可以替我们定义它的结构——`cts/tools/utils/rerun.py`（手工重跑工具）逐层剥开它：

sh cts/tools/utils/rerun.py:31-61,63-87（节选+讲解注释）

```
def parseSuite(suite, parentName):
```

```
if parentName != "":
parentName += '.'
failedCases = []
childSuites = getChildrenWithTag(suite, "TestSuite") # 讲解注释①：第 3 层 TestSuite——一个
instrumentat
ion 包一个节点
for child in childSuites:
for failure in parseSuite(child, parentName + child.getAttribute("name")): #
讲解注释②：TestSuit
e 可嵌套，递归下潜
failedCases.append(failure)
childTestCases = getChildrenWithTag(suite, "TestCase") # 讲解注释③：第 4 层 TestCase = 测试类
for child in childTestCases:
className = parentName + child.getAttribute("name")
for test in getChildrenWithTag(child, "Test"): # 讲解注释④：第 5 层 Test = 单个测试方法
if test.getAttribute("result") != "pass": # 讲解注释⑤：result 属性取值 pass/fail/NOT_EXECUT
ED——只收非 pass
failureName = className + "#" + test.getAttribute("name")
failedCases.append(failureName)
return failedCases
def getFailedCases(resultXml):
failedCases = []
doc = minidom.parse(resultXml)
testResult = doc.getElementsByTagName("TestResult")[0] # 讲解注释⑥：第 1 层
TestResult——根节点，含 Summary
统计
packages = getChildrenWithTag(testResult, "TestPackage") # 讲解注释⑦：第 2 层 TestPackage =
模块（name 属性
如 CtsOsTestCases）
for package in packages:
casesFromChild = parseSuite(package, "")
for case in casesFromChild:
if case not in failedCases:
failedCases.append(case)
return failedCases
def main(argv):
…
failedCases = getFailedCases(resultXml) # 讲解注释⑧：从 XML 提取失败清单
for failure in failedCases:
[className, methodName] = failure.split('#')
command = ctsPath + "/tools/cts-tradefed run singleCommand cts"
if deviceSerial != "":
command += " --serial " + deviceSerial
command += " --class " + className + " --method " + methodName #
讲解注释⑨：层级路径直接翻译成命令行过滤——
print command # 讲解注释⑨：'类#方法' 与 exclude-fi
lter 的粒度同构
os.system(command)
# 讲解注释⑩：这份 89 行的脚本就是"result 层级与命令行的互译"的活例子。Toolchain 另有
# cts-api-coverage 的 CtsReportParser（default 路径 ./test_result.xml）把结果转成
test_result.pb——
# 同一份 XML 是所有下游工具（覆盖报告/重跑/认证提交）的单一事实来源。
```

于是完整结构可以背下来了：**TestResult（根，Summary）→ TestPackage（模块）→ TestSuite（包）→ TestCase（类）→ Test（方法，result 属性）**。每层 name 属性串起来就是报

告与过滤的"全限定名"，例如 `CtsDumpsysHostTestCases android.dumpsys.cts.GfxInfoDumpsysTest #testGfxinfoFramestats`——与第八章 exclude-filter 的四级粒度逐字对应。模块级还记录 preparer 耗时与堆栈快照，logcat/screenshot 存放在同 session 的 `logcat/`、`screenshots/` 下（第二章 2.1 的"结果三件套"）。

9.3 Retry：不是"再跑一遍"，而是"带着旧结果跑一遍"

`cts-tradefed run retry --retry <session_id>` 的实现是一份独立配置：

xml cts/tools/cts-tradefed/res/config/retry.xml:16-24（全文+讲解注释）

```
<configuration description="Runs a retry of a previous CTS session.">
<object type="previous_loader"
class="com.android.compatibility.common.tradefed.result.suite.Pre
viousResultLoader" /> // 讲解注释①：第一步——按 session_id 找到旧 testResult.xml
并解析回内存对象
<test class="com.android.tradefed.testtype.suite.retry.RetryRescheduler" /> //
讲解注释②：第二步——以"重调度
器"身份重新编排被选中的模块
<logger class="com.android.tradefed.log.FileLogger">
<option name="log-level-display" value="WARN" />
</logger>
</configuration>
// 讲解注释③：一个配置两个对象：previous_loader 喂数据，RetryRescheduler 用数据——Retry
与全新运行共用同一条第七章执行链。
```

三步机制（类名均可在预编译 jar 中验证）：**加载**——`PreviousResultLoader`（实现 `ITestSuiteResultLoader`）读取旧 session 的结果 XML；**过滤**——`RetryRescheduler`（配合 `RetryResultHelper` 与 `ResultsPlayer`）把上一轮 **pass 与被 known failures 排除的用例剔除**，只保留 fail/NOT_EXECUTED 的用例集合，策略由 `com.android.tradefed.retry.IRetryDecision/BaseRetryDecision/RetryStrategy` 决定（重试几次、按模块还是按用例）；**重跑**——被选中的模块以过滤后的测试名单重新走 `CompatibilityTestSuite`→shard→`ITestFilterReceiver` 注入的完整链路（第七章 7.2 第 4 级的接口在这里兑现）。CTS 特有的 `RetryFilterHelper` 再叠加一层"已知 flaky 自动重跑"的策略（第五章的设计决策在此落地）。Retry 结束后新旧结果合并成该 session 的最终 testResult.xml——认证提交的是合并后的账。

> **设计洞察 · 排障视角：Retry 三问**
>
> 1. retry 报"session 不存在"——results 目录被清或 session_id 写错，先 `ls results/`；2. retry 跑完仍然 fail 且用例一模一样——失败是稳定复现（非 flaky），回第九章 9.2 定位根因而非继续 retry；3. retry 后 NOT_EXECUTED 模块没被重跑——检查其 preparer 是否仍失败（第六章 6.2 阶段一），环境没修好 Retry 不会替你修。

教科书习题集（分级）

第一级：简答题（检验概念理解）

- **1. CTS 断言的本质是什么？**

  参考答案要点：断言"framework API 的行为与 SDK 文档一致"（第一章）；任何 deviation 都是 fail；通过 CTS = 拿到 GMS 预装权。

- **2. 一个 CTS 设备侧模块由哪两类产物构成？**

  参考答案要点：Android.bp 的 android_test 产出测试 APK（如 CtsOsTestCases.apk）+ test_config 指定的模块配置 XML（第六章 6.1/6.2）；test_suites 声明归属哪些套件。

- **3. cts-tradefed 启动脚本的入口主类是什么？它为 java 进程做了哪些环境准备？**

  参考答案要点：`com.android.compatibility.common.tradefed.command.CompatibilityConsole`（第七章 7.1）；校验 aapt/adb/java 与 JDK 版本、定位 CTS_ROOT 两种形态、拼 classpath、-Xmx6g 加 OOM heap dump。

- **4. CTS 模块配置 XML 的三段结构是什么？**

  参考答案要点：module_controller（跑不跑，如 Sdk30ModuleController）→ target_preparer（环境准备/收尾）→ test（IRemoteTest 执行器）（第六章 6.2）。

- **5. IRemoteTest 为什么是整个调度链的关键接口？**

  参考答案要点：只要求 run(TestInformation, ITestInvocationListener) 一个方法（第七章 7.2）；AndroidJUnitTest/GTest/JarHostTest/Mobly 都实现它，shard 与结果聚合因此对"测试是什么"无感。

- **6. exclude-filter 与 test-arg 的区别？**

  参考答案要点：exclude-filter 是名单制，按"模块 包.类#方法"四级点名（第八章 8.2）；test-arg 是标注制，向指定 runner 注入选项按注解批量排除（如 CtsDownstreamingTest、AsbSecurityTest）。

- **7. known failures 文件的登记纪律是什么？它与 waiver 的关系？**

  参考答案要点：每条 exclude-filter 必须带 b/ bug 号、可登记到方法级、参数化变体（[instant]）单独登记（第八章 8.2）；Google 审批 waiver 后才会进入下一版 known failures——它是认证公平性的账本。

- **8. host 侧模块与设备侧模块的三点差异？**

  参考答案要点：构建类型 java_test_host 产出 jar 而非 APK；runner 用 JarHostTest 而非 AndroidJUnitTest（第九章 9.1）；断言跑在 host JVM，用 ITestDevice/adb 操纵设备，可验证设备自己看不见的东西。

- **9. 写出 testResult.xml 的五层结构。**

参考答案要点：TestResult（根，Summary）→ TestPackage（模块）→ TestSuite（包）→ TestCase（类）→ Test（方法，result 属性）（第九章 9.2）；每层 name 拼出与 exclude-filter 同构的全限定名。

- **10. Retry 的三步机制是什么?**

参考答案要点：PreviousResultLoader 加载旧 session 结果 →RetryRescheduler/RetryResultHelper 剔除 pass 与已排除用例（IRetryDecision/RetryStrategy 定策略）→过滤后名单重新走完整执行链（第九章 9.3）；认证提交的是合并后的结果。

第二级：分析题（检验机制理解）

- **11. 追踪调用链**：从 `cts-tradefed run cts -m CtsOsTestCases` 到 `am instrument` 命令，画出五级执行链并标注每级输入/输出。

参考答案要点：Console 解析命令与计划→CompatibilityTestSuite 枚举 testcases/ 并加载模块 XML→ShardHelper/TestsPoolPoller 切分认领→AndroidJUnitTest（ IRemoteTest+ITestFilterReceiver）→RemoteAndroidTestRunner 组装 am instrument -w -e package android.os.cts ...（第七章 7.2）。

- **12. 失败推演**：若 CtsOsTestCases 的 FilePusher 因辅助 APK 缺失而失败，下游会发生什么?

参考答案要点：preparer 阶段抛异常→模块整体记 fail/NOT_EXECUTED 而非逐用例失败（第九章 9.3 注：preparer 失败 Retry 不重跑用例）→testResult.xml 中该模块无 TestSuite 子节点→retry 也无法救活，必须先修环境（排障手册场景二）。

- **13. 参数化推演**：CtsOsTestCases 声明四种 parameter 后，主 CTS 计划中它最多展开成几个变体？各自的安装/运行形态差异?

参考答案要点：instant_app/multi_abi/secondary_user/multiuser 家族展开（第八章 8.4）；instant 变体按 instant app 安装、multi_abi 32/64 位各跑、secondary_user 切用户；方法级可用 @AppModeFull 等注解退出不适用的变体。

- **14. 跨册串联**：CTS 的"设备侧断言跑在设备进程"与书3 Binder 十跳的交汇点在哪?

参考答案要点：用例调用 framework API（如 PowerManager）经 Binder 进 system_server，断言的是跨进程后的语义；第六章 6.1 的 AIDL srcs（IEmptyService 等）说明部分用例自己起跨进程服务来验证 Binder 行为。

- **15. 分片分析**：runtime-hint 如何影响 shard 均衡？token 机制解决什么问题?

参考答案要点：调度按预估耗时（3m15s，第六章 6.2 注⑧）把模块装入均衡的分片；token（ TelephonyTokenProvider 等，第七章 7.2）把"需 SIM/需通话能力"的模块约束到具备硬件的分片——时长均衡与能力约束是两个正交维度。

- **16. 政策分析**：为什么 CtsDumpsysHostTestCases 的用例会同时出现在 cts-known-failures.xml 中?

参考答案要点：known failures 的粒度是"模块 类#方法"，host 侧模块同样适用（第八章 8.2）；b/23776893 登记的 batterystats/gfxinfo 输出断言依赖系统状态，属环境型失败——排除是政策决定而非代码缺陷。

- **17. 版本推演**：SDK30ModuleController 在 API 29 设备上触发时，结果文件里会出现什么？与 exclude-filter 的 skip 有何区别?

参考答案要点：模块控制器判定不满足则整模块 skip（第六章 6.2 注④），记录为 skipped 而非 fail；exclude-filter 在计划加载期就把用例从名单剔除，两者都不产生失败统计——但前者在运行期、后者在装载期。

第三级：设计题

- **18. 架构演进**：如果让你把"计划=配置叠加"改为"计划=数据库查询"，会失去什么?

参考答案要点：失去配置片段的版本化与可 diff 性（第八章 8.2 洞察）——认证公平性依赖"政策随套件快照发行"；include 链的覆盖语义（后层赋值）在查询模型里要重新定义，且离线解压包将依赖数据库服务。

- **19. 排障方案**：设计"模块 NotExecuted"的分层定位方案。

参考答案要点：第一层看脚本（Console 能否起，第七章 7.1）→第二层看配置加载（7.3 解析异常）→第三层看 preparer 日志（6.2 阶段一/三，APK 安装/push 失败）→第四层看 module_controller 与前置条件（8.3 体检）→每层给一条可直接执行的命令（排障手册场景二）。

- **20. 设计反思**：为 CTS 增加"折叠屏"维度需要改动哪些环节?

参考答案要点：Tradefed 增加参数家族与 handler（jar 中已有 FoldableHandler/FoldableExpandingHandler，第七章/第八章）→模块配置声明 parameter metadata 并过 7.3 的 MANDATORY_PARAMETERS_FAMILY 校验→known failures 按变体登记→host 侧与设备侧 runner 都要能注入过滤。

术语表

术语	解释
CDD	Android 兼容性定义文档；CTS 用例是其条目的代码化断言（第六章 6.3）
CTS	Compatibility Test Suite，兼容性测试套件；framework API 行为的唯一权威（前言）
exclude-filter（排除过滤器）	名单制排除过滤器，粒度四级：模块/包/类/方法（第八章 8.2）
GMS	Google Mobile Services；通过 CTS（及 GTS 等其他门禁）后才能预装的 Google 应用家族（前言）

术语	解释
GTS	Google 专有套件，验证 GMS 闭源组件；接收从主 CTS 划出的 CtsDownstreamingTest 用例（第八章 8.2）
Host 侧测试	断言运行在 host JVM 的用例，通过 ITestDevice/adb 操纵设备，验证设备自身不可见的系统级事实（第九章 9.1）
Instrumentation	Android 官方设备侧执行机制；AndroidJUnitTest 最终组装 am instrument 命令执行（第七章 7.2）
IRemoteTest	Tradefed 测试执行器的最小接口，run(TestInformation, ITestInvocationListener)（第七章 7.2）
ITestSuite	套件级调度抽象；CTS 中的实现是 CompatibilityTestSuite，负责枚举与编排全部模块（第七章 7.2）
JarHostTest	host 侧 runner，反射扫描指定 jar 中的 junit 用例执行（第七章 7.3/第九章 9.1）
Known Failures	known-failures.xml 登记的已确认环境型失败清单，计划加载期整条排除，每条必须带 bug 号（第八章 8.2）
Module Controller	模块配置中的对象（如 Sdk30ModuleController），运行期判定整模块是否执行（第六章 6.2）
MTS	Mainline Test Suite；验证 Mainline 模块，同一测试模块可经 test_suites 同时挂入（第六章 6.1）
Parameterized Module	参数化模块：按 metadata 声明展开为 instant/多 abi/多用户等变体，变体名带方括号后缀（第八章 8.4）
Plan	测试计划：可叠加的配置 XML 片段树，定义跑哪些模块与运行政策（第八章 8.1）
Precondition	前置条件：运行前的设备体检（user 构建/en-US/锁屏/WiFi 等），不过则整个计划 NotExecuted（第八章 8.3）
Retry	重试机制：加载旧 session 结果→剔除 pass→过滤后重跑，最终合并结果（第九章 9.3）
runtime-hint	模块配置中的预估耗时，shard 调度做负载均衡的依据（第六章 6.2/第七章）
session	一次完整运行的结果目录编号（results/<session-id>/），Retry 以它为单位取旧结果（第九章）
Shard	分片：ShardHelper/StrictShardHelper 把模块清单切成 N 份由 TestsPoolPoller 多机认领（第七章 7.2）
STS	Security Test Suite；安全补丁验证套件，接收 AsbSecurityTest 注解用例（第六章/第八章 8.2）
SuiteApkInstaller	套件级 APK 安装 preparer；旧 ApkInstaller 已被配置宪法禁用（第七章 7.3）
target_preparer	模块配置的环境准备角色，含 SuiteApkInstaller/FilePusher/RunCommandTargetPreparer 等（第六章 6.2）

术语	解释
test-arg	按 runner+注解批量注入选项的过滤写法，如向 AndroidJUnitTest 注入 exclude-annotation（第八章 8.2）
testResult.xml	一次运行的结果汇总，五层结构 TestResult/TestPackage/TestSuite/TestCase/Test（第九章 9.2）
Token	分片令牌：SIM/通话能力等硬件条件约束，保证模块落到具备条件的分片（第七章 7.2）
Tradefed	测试调度引擎：Console→Suite→Shard→IRemoteTest 四级流水线的宿主，以预编译 jar 发行（第二章/第七章）
VTS	Vendor Test Suite；验证 HAL 接口实现，与 CTS 分开覆盖厂商层（第三章）
Waiver	豁免：对确认属 Google 侧问题的失败向 Google 申请审批，通过后进 known failures（第四章/第八章 8.2）

排障手册：CTS 认证问题定位

通用路径：1. 看结果 XML 定位失败粒度（第九章 9.2）→ 2. 看 logcat/截图 → 3. 按下述场景对号入座 → 4. 深入各章。

深度实战：CTS 认证的排障场景分析

场景一：设备中途 offline，大量模块中断

- **症状**：控制台刷出 `Device <serial> is not available` 或 `DeviceUnresponsiveException`；testResult.xml 中多个模块 result="fail"、原因为 `com.android.tradefed.device.DeviceNotAvailableException`；剩余模块全部 NOT_EXECUTED。
- **定位命令**：`adb devices -l` 看状态是否 device/offline；`adb -s <serial> shell getprop sys.boot_completed`；`dmesg | grep -i usb` 看 host 侧 USB 掉线记录； Tradefed 日志 `logs/invocation-*/host_log.txt` 搜 `not available`。
- **根因**：USB 线/Hub 供电不稳、设备过热重启、被测 ROM kernel panic、或 adb 端口被其他进程抢占。CTS 长时间高负载（第六章注：8-72 小时）对供电与散热是硬考验。
- **修复**：换短线直连、加散热、`adb reconnect offline`；ROM 引发的要抓 ramdump 单独分析。恢复后对同一 session `run retry --retry <session_id>` 补齐 NOT_EXECUTED 模块（第九章 9.3）。

场景二：模块整体 NotExecuted，一个用例都没跑

- **症状**：testResult.xml 中该模块节点存在但没有 TestSuite 子节点，Summary 统计 skipped；控制台出现 `Failed to install CtsXxx.apk` 或 preparer 抛 `TargetSetupError`。

- **定位命令**：在 host_log.txt 中搜模块名看 preparer 日志；`adb shell pm list packages | grep cts` 检查残留包；`df -h /data` 看存储；手工复现：`adb install -r -t android-cts/testcases/CtsXxx.apk`。
- **根因**：按第六章 6.2 三段结构排查——module_controller 判定不满足（SDK 版本不匹配）；SuiteApkInstaller 安装失败（签名冲突/存储不足/版本被占用）；RunCommandTargetPreparer 或 FilePusher 失败（路径/权限）；计划级前置条件体检不过（第八章 8.3，如非 user 构建）。
- **修复**：清除冲突包（`adb uninstall <pkg>`）、清空 `/data` 空间、刷正确版本 user 构建。preparer 失败不会被 Retry 救活——先修环境再跑（第九章 9.3 排障视角）。

场景三：instrumentation 起不来，报 Permission Denial 或 SecurityException

- **症状**：模块内所有用例秒失败，logcat 出现 `java.lang.SecurityException: Permission Denial: starting instrumentation ... not allowed` 或 `requires android.permission.INSTALL_PACKAGES`。
- **定位命令**：`adb shell getprop ro.build.type`（须 user）；`adb shell dumpsys package android.os.cts | grep -A3 permission`；`adb shell getprop persist.sys.test_harness`（须 false，见第八章 8.3）。
- **根因**：在非认证环境强行跑认证包（未开 test harness 模式的 user 构建拒绝 instrumentation）；或设备厂商改动了权限保护逻辑，导致 framework 偏离（第四章的高频 fail 原因）。
- **修复**：环境问题——确认设备测试笼模式与 CTS 版本匹配；framework 偏离——按第一章断言本质分析是保护逻辑改严还是改松，改严会挡测试进程、改松会过不了其他权限用例，都要修回与 SDK 文档一致。

场景四：host 侧 OOM，Console 进程被杀

- **症状**：`java.lang.OutOfMemoryError: Java heap space`，或进程直接消失但留下 heap dump（第七章 7.1 注释⑨：脚本以 `-Xmx6g -XX:+HeapDumpOnOutOfMemoryError` 启动，OOM 时自动落 dump）。
- **定位命令**：在 CTS 根目录找 `*.hprof`；`jmap -histo <pid> | head -20` 看实例数 Top；host_log.txt 搜 `OutOfMemory`；检查是否开了超大 `-s` 并发与过低的 `max-log-size`（第八章 8.1 注⑧）。
- **根因**：默认堆 6G 被多分片并发 + 大量截图/日志对象挤爆；或引用了会累积结果的自定义 result reporter；host 内存本身不足。
- **修复**：减少并发 shard 数、调大 `-Xmx`（编辑启动脚本）、提高 `max-log-size`/`max-tmp-logcat-file` 换更大 host 盘；仍复现则用 heap dump 找泄漏持有者（多为自定义 listener 持有 InvocationContext）。

场景五：Retry 跑完，结果没有变化

- **症状**：`run retry --retry <session_id>` 输出 `0 modules to run` 或重跑的失败用例与上一轮一字不差。

- **定位命令**：`ls results/` 确认 session 目录存在；看 retry 运行的控制台输出中 `Retrying...` 之后的模块列表是否为空；比对新旧 testResult.xml 的 Summary pass 数。
- **根因**：三种——session_id 写错或 results 目录被清（PreviousResultLoader 找不到旧结果，第九章 9.3）；失败是稳定复现而非 flaky，Retry 按 RetryStrategy 判定后仍全量重跑并再次失败；NOT_EXECUTED 模块的环境根因（场景二）没修，重跑仍卡在 preparer。
- **修复**：先修环境再用 `--retry-type` 选对重试范围——选项值只有 `failed` 与 `not_executed` 两种（经预编译 jar 内 RetryRescheduler 的选项定义核实）；稳定失败回到结果定位（本章开头通用路径），该修代码修代码、该提 waiver 提 waiver（第八章 8.2）。

场景六：用例莫名 fail——flaky 还是真偏离

- **症状**：单个用例 result="fail"，无 crash；同一用例有时通过有时失败；重跑后通过。
- **定位命令**：看结果目录 `logcat/` 中该用例时间窗的输出与 `screenshots/`；`adb shell dumpsys battery`（温度）、`df -h /data`（存储）、`ping`（网络）核对设备状态；用第九章 9.2 的方法从 XML 提取失败全限定名后 **run cts -m <模块> -t <类#方法>** 单独复现 5 次。
- **根因**：环境型 flaky（温度降频、广播队列未空闲——见第六章 6.2 `am wait-for-broadcast-idle` 的用意、网络超时容忍边界）或真实偏离（每次都挂即偏离）。
- **修复**：flaky——改善环境后 `run retry`，多次复现无果且证据指向 Google 侧问题则提 waiver；稳定复现——按第一章"断言=SDK 文档"分析 framework 偏离点，修 framework 或修设备配置，不要绕过测试。

CTS 核心速查表

机制	一句话	源码锚点	所在册
CTS	framework API 行为（App 视角）	cts/tests/tests/	本册
VTS	HAL 接口验证（书14 的 AIDL 逐个过）	system/vts/	本册
GTS	Google 专有（GMS 验证，闭源）	Google 发行	本册
Tradefed	测试调度（选设备→跑→聚合→重试）	cts/tools/tradefed/	本册
断言量	200-300 万（8-72h 可分片并行）	cts/tests/	本册